Johns Hopkins Engineering

Introduction to GPU Programming

Module 11 Lecture 11B An Introduction to OpenCL Contexts

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

Chancellor T. Pascale
Computer Science Department

Overview

2. OpenCL Contexts

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

11.2 OpenCL Contexts

The material in this video is subject to the copyright of the owners of the material and is being provided for educational purposes under rules of fair use for registered students in this course only. No additional copies of the copyrighted work may be made or distributed.

OpenCL Contexts

- Contexts are the heart of any OpenCL application, providing a container for devices, memory objects, and command-queues

- Unlike platforms and devices, which are primarily queried at the beginning of applications, contexts may need to be updated as the program progresses.

Application OpenCL general steps:

1. Query which platforms are present.
2. Query the set of devices for each platform:
    1. Choose to select devices, using clGetDevices(), on specific capabilities.
3. Create contexts from a selection of devices (a context for all devices on the same platform), then with a context you can
    1. Create one or more command-queues
    2. Create programs to run on one or more associated devices
    3. Create a kernel from those programs
    4. Allocate memory buffers and images, either on the host or the device(s)
    5. Write or copy data to/from device
    6. Submit kernels to command-queue for execution

Context Creation

- Two methods for creating contexts:
  - cl context **clCreateContext** (const cl_context_properties * properties, cl_uint num_devices, const cl_device_id * devices, void (CL_CALLBACK * pfn_notify) (const char * errinfo, const void * private_info, size_t cb, void * user_data), void * user_data, cl_int * errcode_ret)
  - cl context **clCreateContextFromType** (const cl_context properties * properties, void (CL_CALLBACK * pfn_notify) (const char * errinfo, const void * private_info, size_t cb, void * user_data), void * user_data, cl_int * errcode_ret)

Context Creation

- properties argument sets the properties based on the platform (at this point limited to CL_CONTEXT_PLATFORM and the platform id to use)
- devices and device type arguments restrict the set of and type of devices
- pfn_notify and user data are used to define a callback function, with user data being the last argument of the callback function.
- cl_int **clGetContextInfo** (cl context context, cl_context_info param_name, size t param_value_size, void * param_value, size t param_value_size_ret)
- This command return specic info about OpenCL context.
- Table 3.5 shows the set of valid values for param_name

Context Retain/Release

- To increment or decrement the reference count for a specific context use the following commands:
  - cl_int clRetainContext (cl_context context)
  - cl_int clReleaseContext (cl_context context)